

ROBUST IMAGE RECOGNITION UNDER REALISTIC IMAGE DEGRADATIONS

**Thinh Nguyen*, Phong Nguyen, Tram La, Trang Tran,
Nam Trinh, Long Do, Tra Le,**
University of Science and Technology of Hanoi (USTH), Hanoi, Vietnam

ABSTRACT

Image classifiers are usually evaluated on clean benchmark images, yet deployed systems often receive photographs that are blurred, noisy, poorly lit, compressed or low in resolution. This study examines how such degradations affect pretrained classifiers and whether convolutional and transformer architectures differ in robustness. Two convolutional neural networks (ResNet-50, ConvNeXt-Tiny) and two vision transformers (DeiT-Small, Swin-Tiny) of comparable size were fine-tuned with an identical recipe on clean Imagenette images and tested on 3,925 held-out images under five degradations (Gaussian blur, Gaussian noise, low light, JPEG compression, resolution downscaling) at three severity levels. All models exceeded 98.5% clean accuracy, but their robustness diverged. DeiT-Small was the most robust, retaining 95.6% of its clean accuracy with a mean corruption error of 47.7 relative to ResNet-50. ConvNeXt-Tiny had the highest clean accuracy (99.7%) but ranked second in mean corruption error. Resolution downscaling was the most damaging degradation: at the heaviest level, accuracy fell to 15.2% for Swin-Tiny and 31.8% for ConvNeXt-Tiny. In our setting, clean accuracy did not predict robustness, and robustness depended more on the specific architecture and degradation type than on the CNN–Transformer distinction.

Keywords: image classification, robustness, image degradation, convolutional neural networks, vision transformers

GitHub: <https://github.com/Xx-Pumpkin-xX/robust-image-recognition>

1 INTRODUCTION

Deep image classifiers now achieve very high accuracy on curated benchmarks such as ImageNet [1; 2]. These benchmarks consist largely of sharp, well-exposed photographs, whereas images captured in practice are frequently degraded by defocus or camera shake, sensor noise, insufficient illumination, lossy compression and low resolution. Such degradations alter pixel statistics without changing the semantic content of the image: a human observer still recognizes the object, but a classifier may not. Studies of common corruptions have shown that models with similar clean accuracy can behave very differently once image quality declines [3; 4]. Robustness to realistic degradations is therefore a prerequisite for deploying recognition systems in settings such as surveillance, mobile photography and autonomous systems, where image quality cannot be controlled.

Architecture choice adds a second dimension to this problem. Convolutional neural networks (CNNs) build representations from local filters with strong spatial inductive biases [2]. Vision transformers instead divide an image into patches and relate them through self-attention [5; 6], while hierarchical variants such as Swin reintroduce locality through windowed attention [7]. Because these designs process local texture and global structure differently, several studies have reported that transformers are more robust than CNNs to some corruptions [8; 9]. Such comparisons, however, are often confounded by differences in model size, pretraining data and training recipe, and robustness is commonly summarized by a single aggregate score. It therefore remains unclear whether robust-

*Team leader. Contact: thinht2410944@usth.edu.vn

ness follows the CNN–Transformer distinction or depends on the specific architecture and type of degradation.

To address this question, we compare two CNNs, ResNet-50 and ConvNeXt-Tiny, and two vision transformers, DeiT-Small and Swin-Tiny, with comparable capacity (21.7M–27.8M parameters) under a shared fine-tuning recipe and test protocol. All four models use weights pretrained on ImageNet-1k and are subsequently fine-tuned on clean Imagenette images with the same recipe, so model-specific fine-tuning choices do not confound the comparison. Their original ImageNet-1k pretraining procedures are not identical; in particular, the ResNet-50 weights come from an older recipe without the strong augmentation used for the other three models, which we discuss as a limitation in Section 8. The models are then tested on images unseen during pretraining under five realistic degradations (Gaussian blur, Gaussian noise, low light, JPEG compression and resolution downscaling), each applied at three severity levels. Robustness is assessed with complementary metrics that capture accuracy loss, relative retention, confidence change, the severity at which performance breaks down, and corruption error relative to a ResNet-50 reference.

The study is guided by three research questions:

- **RQ1:** How does classification accuracy change as image quality decreases under each degradation type?
- **RQ2:** Do CNNs and vision transformers of comparable size differ in robustness, and is any difference consistent across degradation types?
- **RQ3:** Does accuracy on clean images predict robustness under degradation?

This work contributes (i) a controlled comparison of four classifiers of comparable size that share one pretraining dataset, one fine-tuning recipe and one test protocol over 15 degradation conditions; (ii) a multi-metric evaluation that separates absolute performance (accuracy) from relative degradation (retention, corruption error, confidence drop and critical severity point); and (iii) evidence that robustness is architecture- and degradation-specific rather than determined by the CNN–Transformer distinction.

2 RELATED WORK

Robustness to image degradation. Classifiers trained on clean images are known to be sensitive to loss of image quality. Vasiljevic et al. [10] showed that standard CNNs lose substantial accuracy on blurred images, although fine-tuning on blurred data recovers most of it. Dodge and Karam [4] evaluated four networks under blur, noise, contrast reduction, JPEG and JPEG2000 compression and found blur and noise the most harmful. Hendrycks and Dietterich [3] generalized these studies into ImageNet-C, which applies 15 corruption types in four groups (noise, blur, weather and digital) at five severity levels to models trained only on clean images.

ImageNet-C summarizes robustness with the mean Corruption Error (mCE): the errors of each corruption are summed over severity levels, normalized by those of an AlexNet reference and averaged over corruptions [3]. This single score supports model ranking but hides the severity at which a model begins to fail and the classes it confuses. Moreover, all models compared in that study were CNNs.

CNNs versus vision transformers. Several studies have compared the robustness of the two families. Naseer et al. [11] found that vision transformers tolerate occlusion, perturbations and domain shift well, retaining about 60% top-1 accuracy on ImageNet when 80% of the image is randomly occluded. Paul and Chen [9] reported that ViT is more robust than BiT, a strong CNN pretrained on large-scale data. Bhojanapalli et al. [8] found that ViT is at least as robust as ResNet when pretrained on sufficient data. These conclusions depend strongly on the amount of pretraining data, and the compared models usually differ in size and pretraining, with CNNs represented mostly by ResNet-style networks.

More controlled comparisons have since appeared. Bai et al. [12] showed that CNNs can match transformers against adversarial attacks when they adopt the same training recipe, although transformers remained more robust to out-of-distribution inputs. Pinto et al. [13] compared both families under matched settings and found that modern CNNs such as ConvNeXt can be as robust as trans-

formers. These studies, however, report corruption robustness mainly as aggregate scores such as mCE.

Research gaps. Three gaps follow from this literature. First, apart from a few recent studies [12; 13], CNN–Transformer comparisons rarely match model size, pretraining data and training recipe, so observed differences cannot be attributed to architecture alone. Second, these comparisons mostly report accuracy or error rate, while the change in prediction confidence and the severity at which performance collapses receive less attention. Third, results are usually aggregated across severity levels and degradation types, with little per-degradation or per-class analysis. In a small-scale setting, this study addresses the second and third gaps through the multi-metric, per-corruption evaluation introduced in Sections 4 and 5, and addresses the first gap in part by using models of comparable size, a common pretraining dataset and the same fine-tuning recipe.

3 DATASET AND DATA PREPARATION

3.1 DATASET

We use Imagenette2-320 [14], a ten-class subset of ImageNet-1k (ILSVRC 2012) released by fast.ai, in which the shorter side of every image is resized to 320 pixels. The classes are tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball and parachute. Imagenette suits this study for two reasons. Its images share the distribution of the ImageNet data on which all four models were pretrained, and its classes are visually distinct, so accuracy loss under degradation can be attributed to image quality rather than to fine-grained class ambiguity. The dataset contains 13,394 JPEG images, 9,469 in the `train/` folder and 3,925 in the `val/` folder, with each class accounting for 9.1–10.7% of a folder. Grayscale images (173 in `train/`, 69 in `val/`) are converted to RGB on loading. No new images are collected: all degraded images are generated from the official dataset by code.

Version and source. We use version 2 of Imagenette at 320-pixel resolution (`imagenette2-320`), released at <https://github.com/fastai/imagenette> (archive: <https://s3.amazonaws.com/fast-ai-imageclas/imagenette2-320.tgz>). The experiments read a copy hosted on Kaggle (<https://www.kaggle.com/datasets/xbinchen/imagenette2-320>), whose folder structure and image counts match the official release.

3.2 DATA SPLIT

Imagenette `train/` images originate from the ImageNet-1k training set and were therefore seen by all pretrained backbones, whereas `val/` images originate from the ImageNet-1k validation set and were not. Testing on `train/` images would thus overestimate accuracy. We therefore use the entire `val/` folder as the test set and split `train/` per class into 80% for fine-tuning and 20% for validation (Table 1). Validation images serve only to select the best epoch on clean data and are never used to report results. The split is drawn per class with seed 42 after sorting the file names, so it is reproducible, and the three sets share no image.

3.3 PREPROCESSING

All images are converted to RGB, scaled to $[0, 1]$ and normalized with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225) expected by the pretrained models. Training uses random resized cropping and horizontal flipping to reduce overfitting, whereas validation and testing use a deterministic resize and center crop so that results are repeatable (Table 2).

3.4 DEGRADED TEST DATA

Degraded test data are generated from the 3,925 test images using the five degradations and three severity levels defined in Section 4.3.1 (Table 4), giving 15 degraded conditions in addition to the clean condition. Each degradation is applied after the center crop and conversion to $[0, 1]$ and before normalization, after which pixel values are clipped to $[0, 1]$, so every model receives the same

Table 1: Number of images per class. Train and validation are drawn from `train/`; the test set is the official `val/` folder.

Label	Class	Train	Validation	Test
0	tench	770	193	387
1	English springer	764	191	395
2	cassette player	794	199	357
3	chain saw	686	172	386
4	church	753	188	409
5	French horn	765	191	394
6	garbage truck	769	192	389
7	gas pump	745	186	419
8	golf ball	761	190	399
9	parachute	768	192	390
Total		7,575	1,894	3,925

Table 2: Preprocessing applied to each split. Degradations are applied only to the test set, after cropping and before normalization.

Step	Training	Validation and test
Spatial transform	RandomResizedCrop (224), random horizontal flip	Resize (256), CenterCrop (224)
Tensor conversion	Values in $[0, 1]$	Values in $[0, 1]$
Degradation	None	Test set only (Section 3.4), then clipped to $[0, 1]$
Normalization	ImageNet mean and std	ImageNet mean and std

224×224 degraded image. Gaussian noise is the only random degradation; its seed is derived from `CRC32(seed | image path | degradation | level)` with seed 42, so each image always receives the same noise. Degraded images are regenerated at evaluation time rather than stored, and the complete pipeline is provided in the project repository.

4 METHODOLOGY

This section describes the evaluated architectures, the baseline method, the main evaluation method and the comparison strategy. Following ImageNet-C [3], we refer to the synthetic degradations applied to the test images as corruptions. All models are fine-tuned on clean images only (Section 5), and corruptions are applied only at test time, so any loss in performance reflects sensitivity to test-time degradation rather than exposure to those corruptions during fine-tuning.

4.1 EVALUATED ARCHITECTURES

We evaluate two CNNs and two vision transformers of comparable size (Table 3). Within each family, one model is a classical or widely used reference and the other a more recent design. This allows us to examine both between-family and within-family differences rather than treating all CNNs or all transformers as equivalent. The descriptions below focus on properties most relevant to the robustness patterns observed later: how the input is first divided or downsampled, and over what spatial range information is combined.

ResNet-50 [2] is a residual CNN whose blocks learn a residual function $F(x)$ and add it to their input through an identity shortcut. Its stem is an overlapping 7×7 convolution with stride 2 followed by max pooling, and features are combined through stacked 3×3 convolutions.

ConvNeXt-Tiny [15] modernizes ResNet with design choices associated with vision transformers while remaining purely convolutional. Its stem is a non-overlapping 4×4 convolution with stride 4

(a patchify stem), followed by four stages of 7×7 depthwise convolutions with inverted bottlenecks, LayerNorm and GELU activations.

DeiT-Small [6] is a non-hierarchical vision transformer trained on ImageNet-1k with strong augmentation and regularization. It splits the image into non-overlapping 16×16 patches and relates all patches through global self-attention in 12 layers (embedding dimension 384, 6 heads). We use the variant without the distillation token.

Swin-Tiny [7] is a hierarchical vision transformer that embeds non-overlapping 4×4 patches and computes self-attention within local 7×7 windows, which are shifted between consecutive blocks so that information flows across windows. Like ConvNeXt-Tiny, it builds a four-stage multi-scale representation.

Table 3: Comparison of the selected architectures. Parameter counts refer to the original ImageNet-1k models with a 1,000-class head; with the ten-class head used in this study, they range from 21.7M to 27.8M.

Model	Architecture family	Params	FLOPs	Input size
ResNet-50	CNN – Residual network	25.6M	4.1G	224×224
ConvNeXt-Tiny	CNN – Modern ConvNet	28.6M	4.5G	224×224
DeiT-Small	Vision Transformer – Non-hierarchical	22.1M	4.6G	224×224
Swin-Tiny	Vision Transformer – Hierarchical	28.3M	4.5G	224×224

4.2 BASELINE METHOD

The baseline has two roles. First, every model is evaluated on the clean test set, and its clean accuracy (CA) serves as the model-specific reference for retention. On clean images, we also report macro-averaged precision, recall and F1-score over the ten classes, the confusion matrix, and the mean maximum softmax probability (MSP), which provides the reference for confidence drop.

Second, ResNet-50 serves as the baseline model and as the reference for corruption error (Section 4.3.2), while ConvNeXt-Tiny, DeiT-Small and Swin-Tiny serve as comparison models. ResNet-50 is selected for three reasons. First, it is a widely used reference CNN in robustness studies, including ImageNet-C [3], and it is used in controlled CNN–Transformer comparisons such as Bai et al. [12]. Second, among the evaluated architectures it provides the clearest classical convolutional reference: unlike ConvNeXt-Tiny, it does not incorporate design choices such as a patchify stem, large depthwise kernels or LayerNorm that are associated with more recent architectures. Third, using the original ImageNet-C AlexNet reference would introduce a large architecture and capacity mismatch because AlexNet is substantially smaller and is not one of the models evaluated in this study. We therefore normalize corruption error to ResNet-50. Accordingly, the resulting mCE is a ResNet-50-relative measure for this study and should not be interpreted as directly equivalent to the standard AlexNet-normalized ImageNet-C mCE.

The clean-condition reference contributes to RQ3, while the ResNet-50 reference provides a common scale for the corruption-error comparisons used in RQ1 and RQ2.

4.3 MAIN METHOD

Because this project is a comparative robustness study rather than a proposal for a new classifier, the main method is the controlled corruption-evaluation protocol. The same fine-tuned models, including the baseline and comparison models, are evaluated on corrupted versions of the test images and measured using the robustness metrics below. This protocol provides the main evidence for RQ1 and RQ2 and, together with the clean reference of Section 4.2, for RQ3.

4.3.1 CORRUPTION TYPES

We apply five synthetic corruptions designed to approximate common real-world image-quality failures (Table 4). Gaussian blur approximates defocus, Gaussian noise approximates sensor noise, low light approximates underexposure, JPEG compression reproduces lossy storage, and resolution

downscaling approximates the loss of spatial resolution. Resolution downscaling is implemented by area downsampling followed by nearest-neighbor upsampling back to 224×224 , which produces uniform blocks of $k \times k$ pixels. Because 224 is divisible by 2 and 4, the blocks at $k = 2$ and $k = 4$ are exact and aligned with the image grid; at $k = 3$, $224/3$ is not an integer, so the image is reduced to 75×75 cells and the blocks are only approximately 3×3 and not grid-aligned. It is inspired by reduced sensor resolution and pixel-binning effects but is not intended to physically simulate sensor pixel binning. Each corruption has three severity levels, each defined by a fixed parameter value, and Figure 1 shows an example. Corruptions are applied as described in Section 3.4.

Table 4: Corruption types and severity levels.

Group	Type	Level 1	Level 2	Level 3
Blur	Gaussian blur	$\sigma = 1$	$\sigma = 2$	$\sigma = 3$
Noise	Gaussian noise	$\sigma = 0.04$	$\sigma = 0.08$	$\sigma = 0.12$
Illumination	Low light	$(f, \gamma) = (0.6, 1.5)$	$(f, \gamma) = (0.4, 1.9)$	$(f, \gamma) = (0.2, 2.6)$
Digital	JPEG compression	QF = 30	QF = 15	QF = 7
Digital	Resolution downscaling	$k = 2$	$k = 3$	$k = 4$

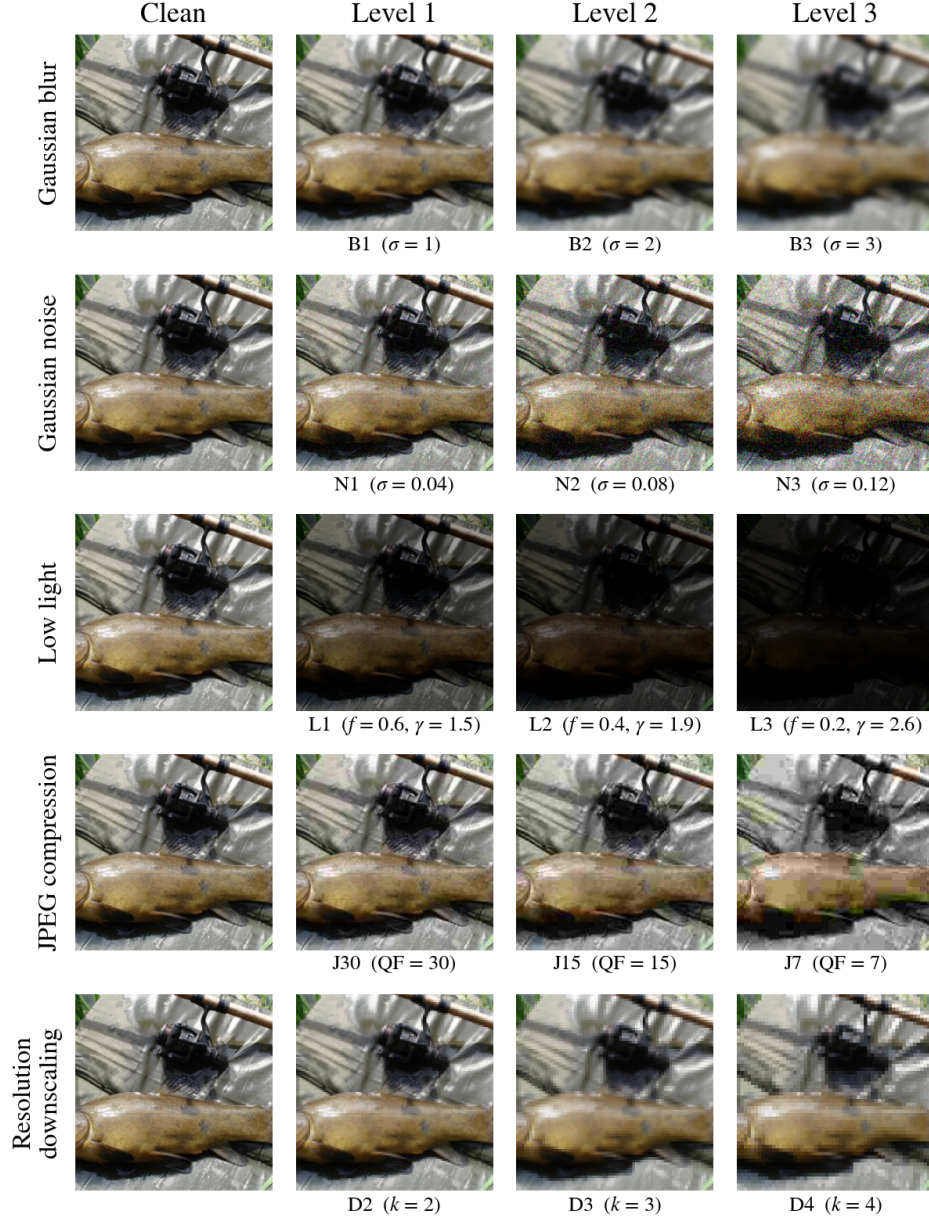


Figure 1: Example of the five corruption types at the three severity levels.

4.3.2 ROBUSTNESS METRICS

Let $A_{s,c}$ denote the accuracy (%) of a model under corruption type $c \in C$ at severity level $s \in \{1, 2, 3\}$, where C is the set of corruption types.

Mean corrupted accuracy. The mean corrupted accuracy (mCA) measures absolute performance under corruption:

$$\text{mCA} = \frac{1}{3|C|} \sum_{c \in C} \sum_{s=1}^3 A_{s,c}. \quad (1)$$

Retention rate. The retention rate (RetR) measures the fraction of clean accuracy that a model retains. For a single condition, $\text{RetR}_{s,c} = A_{s,c}/\text{CA}$; over all conditions,

$$\text{RetR} = \frac{\text{mCA}}{\text{CA}}. \quad (2)$$

Because a model with lower clean accuracy can still obtain a high RetR, RetR is interpreted together with mCA.

Corruption error. Following ImageNet-C [3], corruption error (CE) compares the errors $E_{s,c}^f = 100 - A_{s,c}^f$ of a classifier f with those of a reference model, summed over severity levels. Mean corruption error (mCE) averages CE over corruption types:

$$\text{CE}_c^f = 100 \times \frac{\sum_{s=1}^3 E_{s,c}^f}{\sum_{s=1}^3 E_{s,c}^{\text{ResNet-50}}}, \quad \text{mCE}^f = \frac{1}{|C|} \sum_{c \in C} \text{CE}_c^f. \quad (3)$$

We replace the original AlexNet reference with ResNet-50, the baseline model of Section 4.2. CE is therefore expressed relative to ResNet-50: the baseline scores 100 by definition and lower values indicate fewer corruption errors. Because the numerical values depend on the chosen reference, our mCE should be interpreted as a study-specific relative robustness measure rather than as the standard ImageNet-C mCE.

Confidence drop. The confidence drop (CD) measures how much the mean maximum softmax probability decreases under corruption. For corruption type c at severity level s , with N test images and $p_{i,k}$ the softmax probability of class k for image i ,

$$\text{CD}(c, s) = \frac{100}{N} \sum_{i=1}^N \left[\max_k p_{i,k}^{\text{clean}} - \max_k p_{i,k}^{(c,s)} \right]. \quad (4)$$

The overall CD averages $\text{CD}(c, s)$ over all corruption types and severity levels and is reported in percentage points:

$$\text{CD} = \frac{1}{|C|} \sum_{c \in C} \left[\frac{1}{3} \sum_{s=1}^3 \text{CD}(c, s) \right]. \quad (5)$$

A small confidence drop despite a large accuracy loss may indicate poorly aligned confidence, but CD alone is not a calibration metric.

Critical severity point. The critical severity point (CSP) of a corruption type is the lowest severity level at which $\text{RetR}_{s,c}$ falls below 0.90, that is, the level at which a model retains less than 90% of its clean accuracy. If no level falls below this value, the CSP is reported as “–”. The threshold is a descriptive benchmark choice rather than a calibrated deployment-safety threshold.

4.4 COMPARISON STRATEGY

Results are compared at three levels. Overall comparisons use CA, mCA, RetR, mCE and CD; absolute performance (mCA) and relative stability (RetR and mCE) are read together because they can rank models differently. Per-corruption comparisons use accuracy–severity curves, RetR, CSP and CE to identify which corruption affects which model and at which level performance drops sharply. CD shows whether a model’s average confidence decreases together with its accuracy; a small CD despite a large accuracy loss may indicate poorly aligned confidence. Finally, confusion matrices on clean and corrupted images reveal class-level failure modes that aggregate accuracy hides. In all tables and figures, corruption types and severity levels are abbreviated as in Table 5.

Table 5: Abbreviations of corruption types and severity levels. B, N and L codes give the severity level; J and D codes give the parameter value (JPEG quality factor and block size k).

Group	Type	Level 1	Level 2	Level 3
Blur	Gaussian blur	B1	B2	B3
Noise	Gaussian noise	N1	N2	N3
Illumination	Low light	L1	L2	L3
Digital	JPEG compression	J30	J15	J7
Digital	Resolution downscaling	D2	D3	D4

5 EXPERIMENTAL SETUP

5.1 IMPLEMENTATION AND TRAINING

All models are implemented in PyTorch. ResNet-50, ConvNeXt-Tiny and Swin-Tiny are loaded from torchvision with ImageNet-1k weights (IMAGENET1K_V1), and DeiT-Small is loaded from timm (deit_small_patch16_224). In each model, we replace the 1,000-class classification head with a ten-class linear layer and fine-tune all layers, without freezing any part of the backbone.

All four models are fine-tuned with the same configuration (Table 6). This removes model-specific fine-tuning hyperparameter choices as a source of variation, although the original pretrained weights were produced using different pretraining recipes. The learning rate increases linearly during the first epoch and then follows a cosine decay to zero, with an update after every step. No blur, noise, low-light or compression augmentation is used, so the corruptions in Section 4.3.1 are unseen during fine-tuning. After each epoch, the model is evaluated on the validation set, and the checkpoint with the highest validation accuracy is kept for testing. The selected checkpoints come from epoch 8 for ResNet-50, epoch 5 for ConvNeXt-Tiny, epoch 9 for DeiT-Small and epoch 7 for Swin-Tiny.

Table 6: Training configuration shared by all models.

Setting	Value	Setting	Value
Optimizer	AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.999$	Epochs	10
Peak learning rate	1×10^{-4}	Loss function	Cross-entropy
Weight decay	0.05	Numerical precision	Mixed precision (FP16)
LR schedule	1-epoch linear warm-up, cosine decay to 0	Data augmentation	As in Table 2
Batch size	64 (118 steps per epoch)	Checkpoint selection	Highest validation accuracy

5.2 SETUP 1: BASELINE VERSUS COMPARISON MODELS

All four models are evaluated on the 3,925 clean test images. The baseline model, ResNet-50, is compared with ConvNeXt-Tiny, DeiT-Small and Swin-Tiny in terms of clean accuracy, macro-averaged precision, recall and F1-score, and clean loss. This setup establishes the clean-performance reference and the clean-accuracy ranking used for RQ3. Results are reported in Section 6.1.

5.3 SETUP 2: MAIN RESEARCH EXPERIMENT

The selected checkpoint of each model is evaluated on the 3,925 test images under 16 conditions: the clean condition and the 15 corrupted conditions of Table 4, giving 64 model-condition evaluations in total. Each condition is generated once and passed to all four models, so every model receives exactly the same input images (Section 3.4). Inference uses the same mixed precision as training. For every model and condition, we store the softmax output of each test image and compute all metrics of Section 4.3.2 from these outputs with NumPy and scikit-learn. The CSP threshold of 0.90 was fixed in the experiment configuration before the test run. This setup provides the main evidence for RQ1 and, together with Setup 1, for RQ3. Results are reported in Sections 6.1 and 6.2.

5.4 SETUP 3: ROBUSTNESS ANALYSIS

We analyze robustness per corruption type through retention rates and critical severity points and compare corruption errors per type to expose worst-case behavior hidden by mCE. At the most damaging condition (D4), we inspect confusion matrices and individual misclassified images to identify systematic failure modes. This setup answers RQ2 and refines RQ1 by examining how robustness depends on the corruption type and architectural design. Results are reported in Sections 6.3, 6.4 and 7. No ablation study is performed; Section 8 discusses this as a limitation.

5.5 COMPUTATIONAL ENVIRONMENT

All experiments are run on Kaggle Notebooks with a single NVIDIA Tesla T4 GPU (16 GB), using Python 3.12, PyTorch 2.10.0, torchvision 0.25.0 and timm 1.0.26. One training epoch, including validation, takes on average 39 s for ResNet-50, 43 s for ConvNeXt-Tiny, 30 s for DeiT-Small and 56 s for Swin-Tiny, so training all four models takes about 28 minutes. All random number generators are seeded with 42. Because cuDNN autotuning is enabled and interrupted Kaggle sessions are resumed from per-epoch checkpoints, retraining may change the results slightly.

6 RESULTS AND DISCUSSION

This section reports the results of the three setups: clean and overall corrupted performance (Section 6.1), accuracy across severity levels (Section 6.2), per-corruption robustness (Section 6.3), and a discussion of architectural effects (Section 6.4).

6.1 OVERALL ROBUSTNESS BENCHMARK

Table 7 summarizes the main metrics on the clean and corrupted test sets.

Table 7: Overall results. Accuracy, precision, recall, F1, mCA and RetR are in %; clean loss is the cross-entropy on clean images; CD is in percentage points; mCE is relative to ResNet-50 (= 100). The best value in each column is in bold.

Model	Clean acc.	Precision	Recall	F1	Clean loss	mCA	RetR	CD	mCE
ResNet-50	98.55	98.54	98.55	98.54	0.045	89.80	91.13	6.93	100.0
ConvNeXt-Tiny	99.69	99.70	99.69	99.69	0.015	90.77	91.05	5.57	66.2
DeiT-Small	99.21	99.22	99.21	99.21	0.026	94.83	95.58	2.96	47.7
Swin-Tiny	99.46	99.46	99.46	99.46	0.017	89.17	89.64	5.68	80.3

On clean images (Setup 1), ConvNeXt-Tiny achieves the highest accuracy (99.69%) and the lowest clean loss (0.015), although all four models exceed 98.5% and differ by only about one percentage point. Under corruption (Setup 2), DeiT-Small is the most robust model: it obtains the highest mCA (94.83%) and RetR (95.58%), the smallest confidence drop (2.96 percentage points) and the lowest mCE (47.7). The models differ by more than five points in mCA, so corruption robustness separates them much more strongly than clean accuracy does. The ranking also changes: ConvNeXt-Tiny retains no more of its clean accuracy (91.05%) than ResNet-50 (91.13%), whereas DeiT-Small, only third in clean accuracy, has the highest retention.

The two aggregate robustness metrics can also disagree. Swin-Tiny has a lower mCA than ResNet-50 (89.17% versus 89.80%) but also a lower mCE (80.3 versus 100). mCA averages absolute accuracy across all corrupted conditions, so Swin-Tiny’s large drop under resolution downscaling strongly affects the mean. In contrast, mCE first normalizes each corruption by the corresponding errors of ResNet-50 and then averages across corruption types, so Swin-Tiny’s substantially smaller errors under several other corruptions offset its poor resolution-downscaling performance (Table 9). The two metrics therefore capture different aspects of robustness and should be interpreted together.

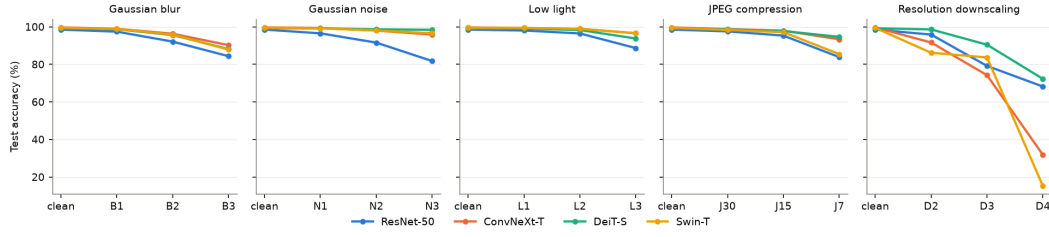


Figure 2: Test accuracy of the four models on clean images and at the three severity levels of each corruption type. Codes follow Table 5.

6.2 ACCURACY ACROSS SEVERITY LEVELS

Figure 2 shows how the accuracy of each model changes as severity increases from level 1 to level 3. For every model and corruption type, accuracy does not increase with severity. ResNet-50 declines most under Gaussian blur, Gaussian noise, low light and JPEG compression, for example from 98.55% to 81.78% at N3. Resolution downscaling follows a different pattern: at D4, accuracy drops sharply to 31.82% for ConvNeXt-Tiny and 15.24% for Swin-Tiny, whereas ResNet-50 (68.20%) and DeiT-Small (72.28%) degrade more gradually. For Swin-Tiny, the drop is concentrated at the final level, from 83.69% at D3 to 15.24% at D4. ConvNeXt-Tiny, DeiT-Small and Swin-Tiny remain above 93% at all levels of Gaussian noise and low light.

6.3 RETENTION RATE, CRITICAL SEVERITY POINT AND CORRUPTION ERROR

Table 8 summarizes the mean retention rate and critical severity point of each model for every corruption type, and Table 9 reports the corresponding corruption errors.

Table 8: Mean retention rate (%) per corruption type, with the critical severity point in parentheses (“–” = retention never falls below 0.90). The best value in each column is in bold.

Model	Gaussian blur	Gaussian noise	Low light	JPEG compression	Resolution downscaling
ResNet-50	92.6 (3)	91.3 (3)	95.8 (3)	93.6 (3)	82.3 (2)
ConvNeXt-Tiny	95.5 (–)	98.0 (–)	98.6 (–)	97.0 (–)	66.1 (2)
DeiT-Small	94.9 (3)	99.5 (–)	97.8 (–)	97.8 (–)	87.8 (3)
Swin-Tiny	94.7 (3)	98.5 (–)	98.9 (–)	94.2 (3)	62.0 (1)

DeiT-Small has the highest retention under Gaussian noise (99.5%), JPEG compression (97.8%) and resolution downscaling (87.8%), while ConvNeXt-Tiny leads under Gaussian blur (95.5%) and Swin-Tiny under low light (98.9%). Resolution downscaling is the most damaging corruption for every model: ConvNeXt-Tiny and Swin-Tiny retain only 66.1% and 62.0% and reach their CSP at levels 2 and 1, respectively. DeiT-Small does not cross the threshold until level 3, whereas ConvNeXt-Tiny stays above it for four of the five corruption types but crosses it at level 2 under resolution downscaling.

Table 9: Corruption error (CE) per corruption type and mean corruption error (mCE), relative to ResNet-50 (= 100). Lower is better; the best value in each column is in bold.

Model	Gaussian blur	Gaussian noise	Low light	JPEG compression	Resolution downscaling	mCE
ResNet-50	100.0	100.0	100.0	100.0	100.0	100.0
ConvNeXt-Tiny	55.4	22.4	30.7	42.5	180.2	66.2
DeiT-Small	67.2	12.4	52.4	38.6	68.0	47.7
Swin-Tiny	66.8	20.7	29.6	81.9	202.5	80.3

Table 9 shows that the mean can conceal worst-case behavior. ConvNeXt-Tiny and Swin-Tiny have lower mCE than ResNet-50 (66.2 and 80.3) and make substantially fewer errors under Gaussian noise and low light, but under resolution downscaling their errors are 1.8 and 2.0 times those of ResNet-50. DeiT-Small is the only model whose CE remains below 100 for every corruption type. Models should therefore be compared per corruption type rather than ranked solely by one clean or averaged score.

6.4 DISCUSSION: ARCHITECTURE AND ROBUSTNESS

The results do not support a simple CNN-versus-Transformer ranking. DeiT-Small is the most robust model overall, but Swin-Tiny, the other transformer, has the lowest mCA and RetR, while ConvNeXt-Tiny, a CNN, ranks second in mCE. Robustness therefore appears to depend more on the particular architecture and corruption type than on the CNN–Transformer family label alone.

The clearest pattern concerns resolution downscaling. ConvNeXt-Tiny and Swin-Tiny, the two models with a non-overlapping 4×4 stem (Section 4.1), are also the two models whose accuracy collapses most severely at D4, and they default to the same two classes (Section 7.1). At D4, $224/4 = 56$ is an integer, so the 4×4 blocks coincide exactly with the 4×4 stem patches: each stem patch receives a single uniform color and all within-patch texture is lost. At D2, each stem patch contains exactly four distinct 2×2 blocks, and at D3 the approximately 3×3 blocks are not aligned with the stem grid (Section 4.3.1), so each stem patch still spans several distinct values, and Swin-Tiny changes comparatively little between these two levels (86.19% and 83.69%) before falling to 15.24% at D4. ResNet-50, whose overlapping 7×7 stem spans neighboring blocks, and DeiT-Small, whose 16×16 patches contain multiple D4 blocks, degrade more gradually.

Stem alignment alone, however, does not explain the full pattern. At D2, where each 4×4 stem patch still contains several distinct image blocks, Swin-Tiny (86.19%) and ConvNeXt-Tiny (91.64%) are already below ResNet-50 (95.87%) and DeiT-Small (98.62%). Severe JPEG compression provides another useful counterexample: QF = 7 creates strong block artifacts aligned with the image grid, yet ConvNeXt-Tiny still achieves 93.35% accuracy while Swin-Tiny falls to 85.48%. The shared 4×4 stem is therefore a plausible contributor to the D4 collapse rather than a complete causal explanation.

Outside resolution downscaling, ConvNeXt-Tiny and Swin-Tiny are among the most robust models, particularly under Gaussian noise and low light. Their severe weakness at D4 is therefore corruption-specific rather than evidence of generally poor robustness. These architectural explanations remain hypotheses consistent with the observed results rather than causal conclusions, because the models also differ in their original pretraining recipes and each design is represented by only one model.

7 ERROR AND QUALITATIVE ANALYSIS

This section examines how the models fail under severe degradation and how visual severity relates to model sensitivity.

7.1 SINK-CLASS COLLAPSE UNDER SEVERE DEGRADATION

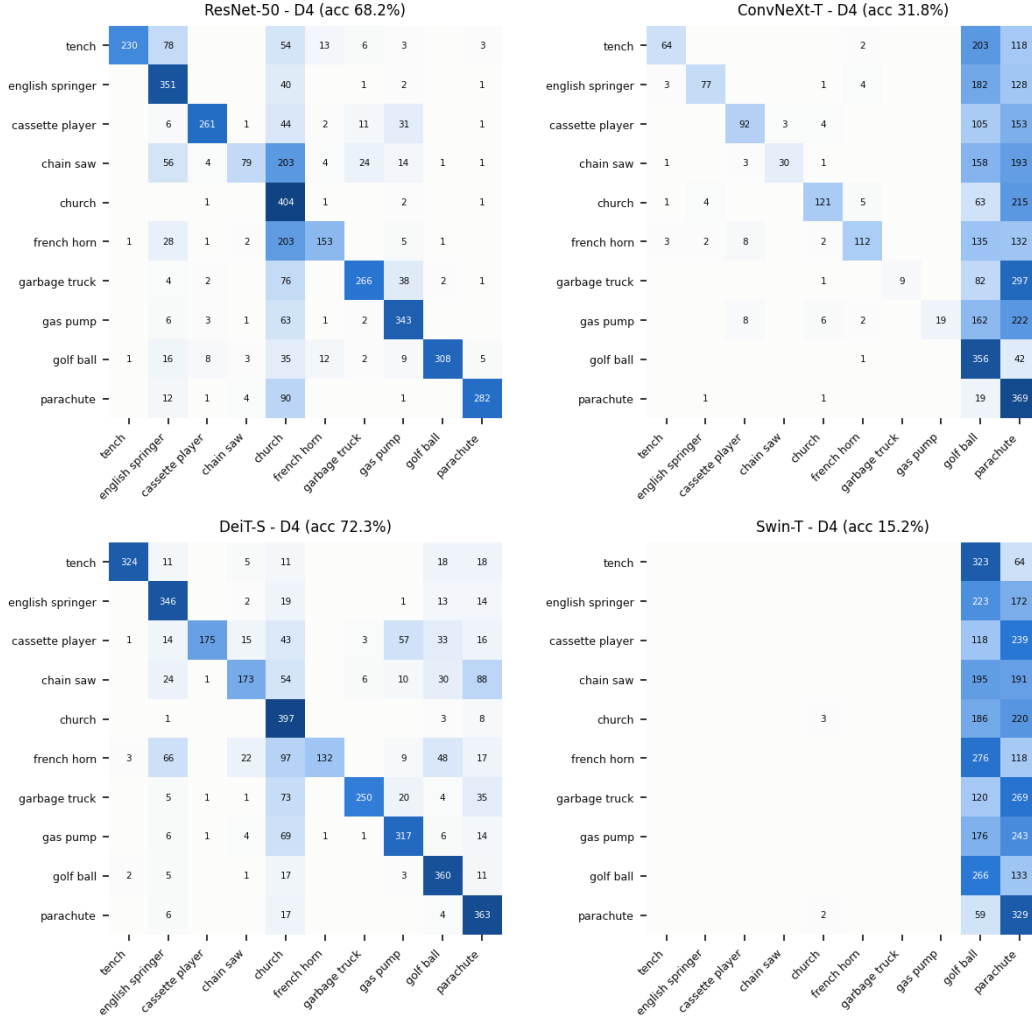


Figure 3: Confusion matrices of the four models at D4 (resolution downscaling, level 3).

At D4, errors do not spread evenly across classes but collapse onto a few sink classes (Figure 3). Swin-Tiny assigns 99.9% of all test images to *golf ball* or *parachute*, so its 15.2% accuracy comes almost entirely from these two classes. ConvNeXt-Tiny shows the same two sink classes, which receive 84.9% of its predictions. ResNet-50 and DeiT-Small instead default mainly to *church*, which absorbs 65% and 37% of their errors, respectively. The sink classes therefore do not follow the CNN-Transformer divide but instead group ConvNeXt-Tiny with Swin-Tiny, consistent with their shared architectural characteristics discussed in Section 6.4. Aggregate accuracy hides this systematic prediction bias.

7.2 VISUAL SEVERITY VERSUS MODEL SENSITIVITY

Visual severity is a poor predictor of model difficulty (Figure 1). At level 3, low light makes the example image appear almost black, yet Swin-Tiny still reaches 96.6% accuracy. Resolution downscaling at the same level looks only moderately pixelated but reduces the same model to 15.2%, and it is the most damaging corruption for all four models. Gaussian noise shows a similar mismatch: it is clearly visible at level 3, yet DeiT-Small loses less than one percentage point (from 99.21%

to 98.42%), while ResNet-50 drops to 81.78%. How severe a degradation looks to the human eye therefore does not reliably indicate how damaging it will be to a classifier.

7.3 EXAMPLE ERROR CASES

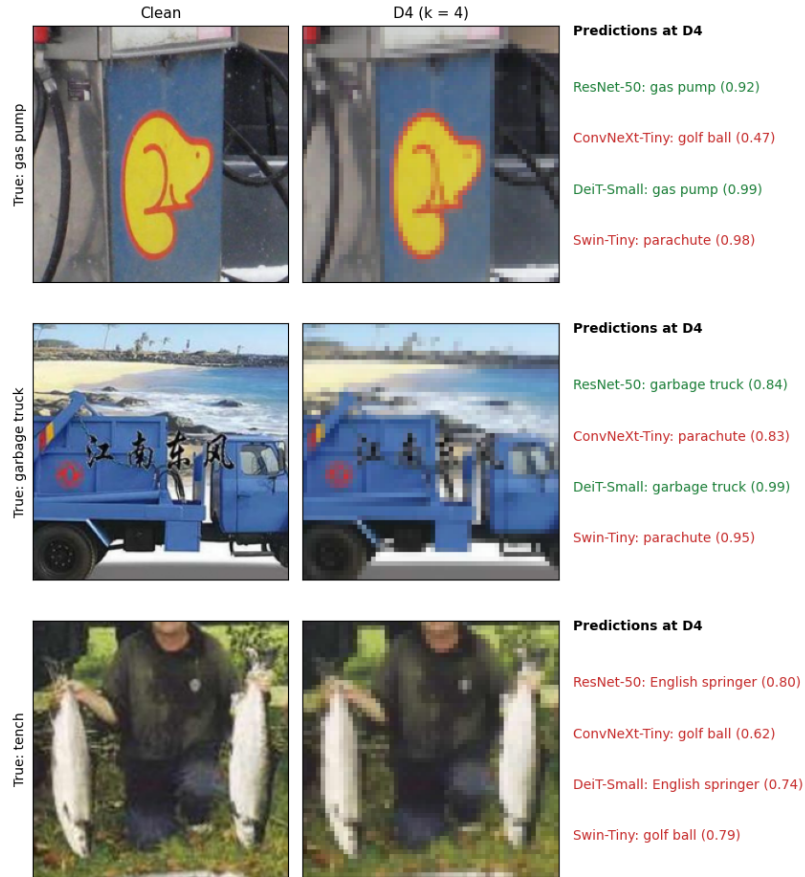


Figure 4: Three test images at D4 with the prediction and confidence of each model. Green: correct; red: incorrect.

Figure 4 shows three test images at D4. A gas pump image is correctly classified by ResNet-50 and DeiT-Small, with confidence 0.92 and 0.99, but Swin-Tiny predicts *parachute* with confidence 0.98 and ConvNeXt-Tiny predicts *golf ball*. A garbage truck image follows the same pattern. A tench image is misclassified by all four models: as *English springer* by ResNet-50 and DeiT-Small, and as *golf ball* by ConvNeXt-Tiny and Swin-Tiny.

These errors are not uniformly low-confidence. At D4, the mean maximum softmax probability among incorrect predictions ranges from 0.58 for ResNet-50 to 0.64 for DeiT-Small and Swin-Tiny, while several individual failures in Figure 4 remain highly confident. Confidence alone therefore does not reliably identify these failures. The examples are also consistent with the architectural pattern discussed in Section 6.4: when D4 produces uniform 4×4 blocks, ConvNeXt-Tiny and Swin-Tiny lose fine within-block texture and frequently fall back on the same sink classes. This observation is consistent with, but does not by itself establish, the stem-alignment hypothesis.

8 CONCLUSION AND LIMITATIONS

Across four architectures of comparable size and five synthetic corruption types, accuracy does not increase with corruption severity for any model, and resolution downscaling causes the largest performance loss.

For RQ1, robustness varies strongly across corruption types. Gaussian noise and low light reduce the accuracy of ConvNeXt-Tiny, DeiT-Small and Swin-Tiny by less than six percentage points even at level 3, whereas resolution downscaling reduces accuracy by as much as 84 percentage points. For RQ2, we find no consistent advantage for either CNNs or vision transformers. DeiT-Small achieves the best overall robustness ($mCE = 47.7$), whereas Swin-Tiny is the most vulnerable to resolution downscaling. ResNet-50 is also more robust to this corruption than ConvNeXt-Tiny and Swin-Tiny despite its lower clean accuracy. The shared 4×4 stem of the latter two models provides one plausible, but incomplete, explanation for this behavior. For RQ3, clean accuracy does not predict robustness in our setting: ConvNeXt-Tiny has the highest clean accuracy, whereas DeiT-Small achieves the highest retention (95.6%) and the smallest confidence drop. Because all four models already achieve very high clean accuracy and differ by only about one percentage point, this result should be regarded as indicative rather than conclusive. The qualitative analysis further shows that severe degradation can concentrate predictions into a small number of sink classes, a failure mode hidden by aggregate accuracy alone.

Several limitations remain. The models were pretrained using different recipes, which confounds architectural comparisons; in particular, the ResNet-50 IMAGENET1K_V1 weights were trained without the stronger augmentation used for the other three models, so part of its lower robustness may reflect pretraining rather than architecture. Each architectural design is represented by only one model, with two models per family. All results come from a single training run, so the study does not measure variation across independent training seeds, and differences between models are not tested with a paired significance test. No ablation study was performed, so the stem-alignment explanation remains a hypothesis rather than a causal result. The benchmark contains only ten relatively easy Imagenette classes, which leads to clean accuracies close to 100%. The corruptions are also synthetic approximations: low light is simulated without additional sensor noise, JPEG compression is applied to images that are already JPEG-encoded, and resolution downscaling does not physically model sensor binning. Finally, confidence drop measures changes in average prediction confidence rather than calibration, and the 0.90 CSP threshold is a descriptive benchmark choice.

Future work should evaluate additional architectures and multiple training seeds, repeat the ResNet-50 comparison using weights obtained from a more modern pretraining recipe, and add calibration measures such as expected calibration error. The stem-alignment hypothesis could be tested directly by shifting the D4 block grid relative to the image origin and by using downscaling factors whose block sizes are not multiples of the 4×4 stem size. More realistic degradation models, real degraded-image datasets and degradation-aware training would further test whether the patterns observed here extend beyond the controlled Imagenette setting.

REFERENCES

- [1] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. doi: 10.1109/CVPR.2009.5206848.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015. URL <https://arxiv.org/abs/1512.03385>.
- [3] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations, 2019. URL <https://arxiv.org/abs/1903.12261>.
- [4] Samuel Dodge and Lina Karam. Understanding how image quality affects deep neural networks, 2016. URL <https://arxiv.org/abs/1604.04004>.
- [5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021. URL <https://arxiv.org/abs/2010.11929>.

- [6] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention, 2021. URL <https://arxiv.org/abs/2012.12877>.
- [7] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows, 2021. URL <https://arxiv.org/abs/2103.14030>.
- [8] Srinadh Bhojanapalli, Ayan Chakrabarti, Daniel Glasner, Daliang Li, Thomas Unterthiner, and Andreas Veit. Understanding robustness of transformers for image classification, 2021. URL <https://arxiv.org/abs/2103.14586>.
- [9] Sayak Paul and Pin-Yu Chen. Vision transformers are robust learners, 2021. URL <https://arxiv.org/abs/2105.07581>.
- [10] Igor Vasiljevic, Ayan Chakrabarti, and Gregory Shakhnarovich. Examining the impact of blur on recognition by convolutional networks, 2017. URL <https://arxiv.org/abs/1611.05760>.
- [11] Muzammal Naseer, Kanchana Ranasinghe, Salman Khan, Munawar Hayat, Fahad Shahbaz Khan, and Ming-Hsuan Yang. Intriguing properties of vision transformers, 2021. URL <https://arxiv.org/abs/2105.10497>.
- [12] Yutong Bai, Jieru Mei, Alan Yuille, and Cihang Xie. Are transformers more robust than CNNs?, 2021. URL <https://arxiv.org/abs/2111.05464>.
- [13] Francesco Pinto, Philip H. S. Torr, and Puneet K. Dokania. An impartial take to the CNN vs transformer robustness contest, 2022. URL <https://arxiv.org/abs/2207.11347>.
- [14] J. Howard. Imagenette, 2019. URL <https://github.com/fastai/imagenette>.
- [15] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s, 2022. URL <https://arxiv.org/abs/2201.03545>.

A TEAM CONTRIBUTIONS

Table 10: Contribution of each team member.

Student name	Contributed aspects	Details
Nguyen Truong Thinh	Team leading, research on corruption methods, coding support, report review and writing, presentation slides	As team leader, I coordinated the tasks and timeline of the team. I collaborated with my teammates to research the corruption types and their severity levels, and I supported Phong in implementing the experiments on Kaggle. I reviewed and edited the entire report, checking its writing style, the consistency between sections, and all reported numbers against the code outputs. I also prepared the presentation slides together with Tra.
Nguyen Hai Phong	Research on evaluation methods, research on corruption methods, main implementation on Kaggle, GitHub repository and documentation	I collaborated with my teammates to research evaluation methods and corruption types. I implemented the main experimental pipeline on Kaggle, including the data split, corruption generation, model fine-tuning and evaluation. I also prepared the project's GitHub repository, including the README and the instructions needed to reproduce the results.
La Thi Bao Tram	Research on evaluation methods, research on corruption methods, report writing	I searched for and read papers related to the topic, which supported the Related Work section. I collaborated with my teammate Trang to research evaluation metrics suitable for 10-class classification tasks, covering both clean and corrupted images. I also researched the corruption types Gaussian blur, JPEG compression and low light. In addition, I wrote the Related Work, Dataset and Data Preparation, and the evaluation-metric part of Section 4.
Tran Thi Thu Trang	Research on evaluation methods, research on corruption methods, report writing	I collaborated with my teammate to research methods to evaluate and compare the performance and robustness of each model, as well as research on corruption types. I proposed the resolution-downscaling corruption, inspired by pixel binning used in mobile-phone imaging, because it is simple and computationally inexpensive to reproduce. I also helped with report writing: I wrote Subsections 4.2 and 4.4.
Trinh Phuong Nam	Research on CNN architectures (ResNet-50 and ConvNeXt-Tiny), report writing	I researched the ResNet-50 and ConvNeXt-Tiny models and illustrated the architectures of both models. I also collaborated with teammates to write Section 6 based on the experimental results provided by the team.
Do Hoang Long	Research on the Swin Transformer architecture, report writing	I researched the Swin Transformer architecture and the Swin-Tiny model. I also wrote the initial drafts of Sections 7 and 8 based on the experimental results and figures provided by the team.
Le Thanh Tra	Research on the DeiT-Small architecture, 4-model comparison table, report writing	I researched the operating principles of the DeiT-Small model and wrote its description in Section 4.1 (Evaluated Architectures). I also helped collect the model specifications (parameters, FLOPs, architecture family) to construct Table 3, the comparison of the selected architectures.